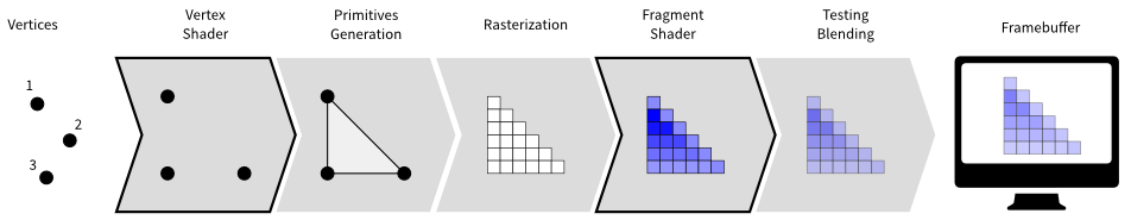
OpenGL – TP Matrices de transformation et projection

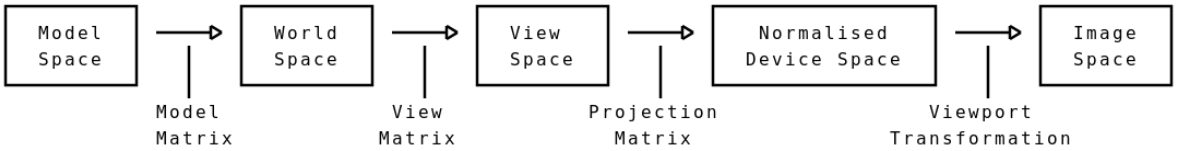
Malek.Bengougam@gmail.com

Comme nous l'avons vu, le trajet de points (ou vertices) dans le pipeline du GPU passe par plusieurs étapes (stages en anglais).



Les éléments programmables que sont les *Vertex Shade*r et *Fragment Shader* nous permettent de customiser le rendu. La manipulation des *vertices* se fait logiquement via un *Vertex Shader*.

Un des aspects fondamentaux de l'infographie consiste à simuler une scène en 3D ou en 2D et à l'afficher dans un canevas en 2D (dans une fenêtre ou en plein écran). La difficulté est ici que les unités peuvent être différentes entre la scène 3D ou 2D, le repère du GPU et celui du *viewport*.

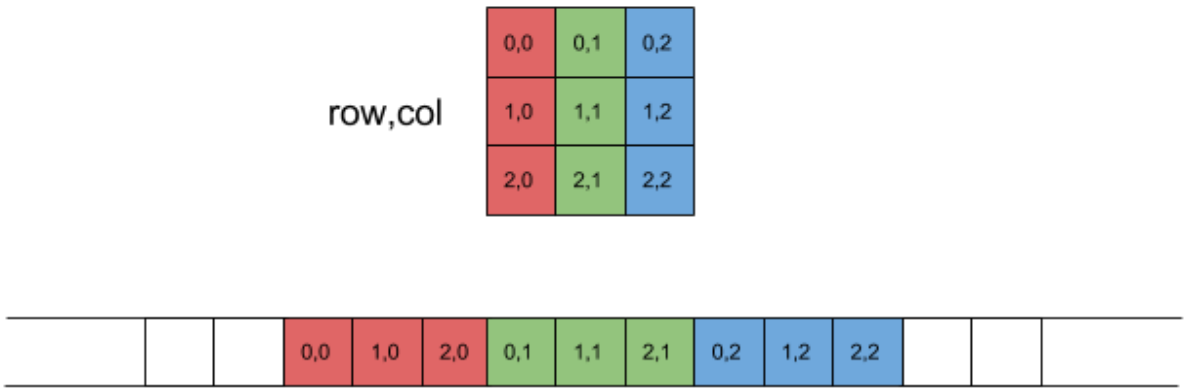


Il est coutume en infographie de simuler plusieurs espaces différents afin faciliter les opérations sur les modèles, la scène ou la caméra.

Dans ce document nous allons nous limiter à l'étude des repères monde et clipping.

Nous allons là encore utiliser des matrices pour représenter ces projections, d'une part car elles sont une représentation simple et compacte de ces transformations, mais également parce que les matrices en OpenGL s'utilisent comme des variables uniformes, donc avec le même mécanisme que l'on a vu dans les cours précédents.

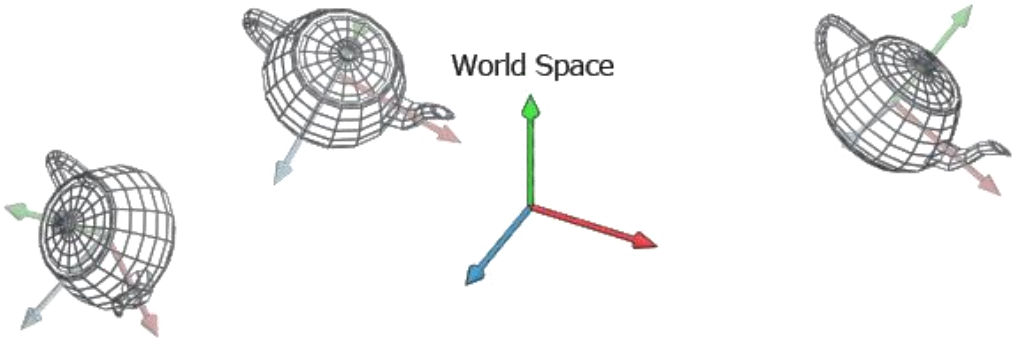
Ne pas oublier qu'OpenGL suit la convention mathématique et donc qu'il s'agit de matrices colonnes. CEPENDANT, en mémoire, le stockage s'effectue colonne après colonne, d'abord la première colonne, puis la seconde etc…



Les matrices que nous allons étudier sont des matrices carrées, 4 lignes, 4 colonnes, en coordonnées homogènes. C'est à dire des coordonnées 3D augmentées d'une 4ème coordonnée afin de pouvoir combiner transformations linéaires et transformations affines.

Repère du monde

La première transformation que nous allons étudier est la transformation dite "local vers monde" (local to world) aussi appelée transformation monde ou modèle (world transform ou model transform). Son rôle est de placer un modèle dans une scène représentant un monde.



La matrice représentant cette transformation est appelée matrice monde (world matrix) ou matrice modèle (model matrix), vous verrez parfois également local-to-world matrix. Bref, l'essentiel est de retenir que le but de cette matrice est de convertir les coordonnées exprimées en repère local (celui du modèle) vers le repère global ou monde (celui de la scène donc).

Nous allons donc former une matrice composée de la concaténation d'une matrice d'homothétie (scale), d'une matrice de rotation, et d'une matrice de translation. Rappelez-vous, mathématiquement on combine les matrices en partant de la droite vers la gauche.

$$\begin{bmatrix} 1 & 0 & 0 & Translation.x \\ 0 & 1 & 0 & Translation.y \\ 0 & 0 & 1 & Translation.z \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos(\theta) & -\sin(\theta) & 0 \\ 0 & \sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} Scale.x & 0 & 0 & 0 \\ 0 & Scale.y & 0 & 0 \\ 0 & 0 & Scale.z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

La combinaison de ces transformations dans cet ordre; scale, puis rotation, puis translation; forme une transformation dite "rigid-body" qui forme la transformée monde (world transform).

Cette matrice "rigide" peut se construire directement de la façon suivante :

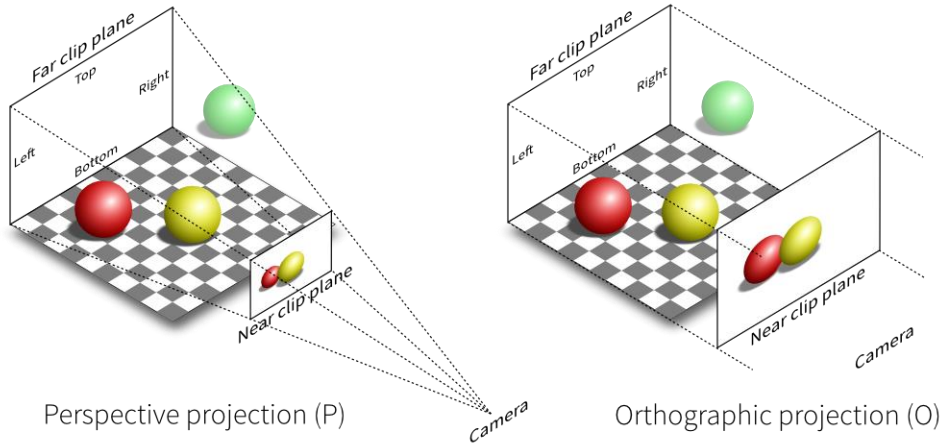
$$\begin{bmatrix} Transform_XAxis.x & Transform_YAxis.x & Transform_ZAxis.x & Translation.x \\ Transform_XAxis.y & Transform_YAxis.y & Transform_ZAxis.y & Translation.y \\ Transform_XAxis.z & Transform_YAxis.z & Transform_ZAxis.z & Translation.z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

On sait que les 3 premières colonnes représentent les transformations sur les axes. Ainsi dans les trois premières colonnes on va retrouver les rotations et homothéties (scale) des axes.

Repère de clipping

Aussi appelé repère “après projection” (*post projection space*) et qui est en pratique le repère de gl_Position. En effet, le rasterizer va procéder à une étape supplémentaire appelée “division perspective” (perspective divide) afin de produire des coordonnées dans le repère normalisé NDC.

Nous allons étudier deux types de projection qui sont les projections orthographiques et perspectives. Comme nous venons de le dire, ces projections ont pour but de convertir les coordonnées exprimées en repère caméra (ou monde, si pas de caméra) vers le repère de clipping qui est l’étape avant la rasterization en NDC.



Projection orthographique

Cette projection consiste à réduire un repère 3D (ou 2D) afin de faire converger les “rayons” de manière parallèle vers un plan à 2 dimensions.

Cette matrice de projection permet de convertir des unités du monde (en pixels ou unités arbitraires) vers les unités normalisées du GPU (NDC). Elle se présente comme le produit de deux opérations, d’abord un “*scale*” (mise à l’échelle) non uniforme dans la diagonale, ceci afin d’adapter chaque axe de manière différencié. Puis une translation (quatrième colonne) négative spécifiant le décalage de l’origine par rapport à l’origine du repère NDC qui est centré.

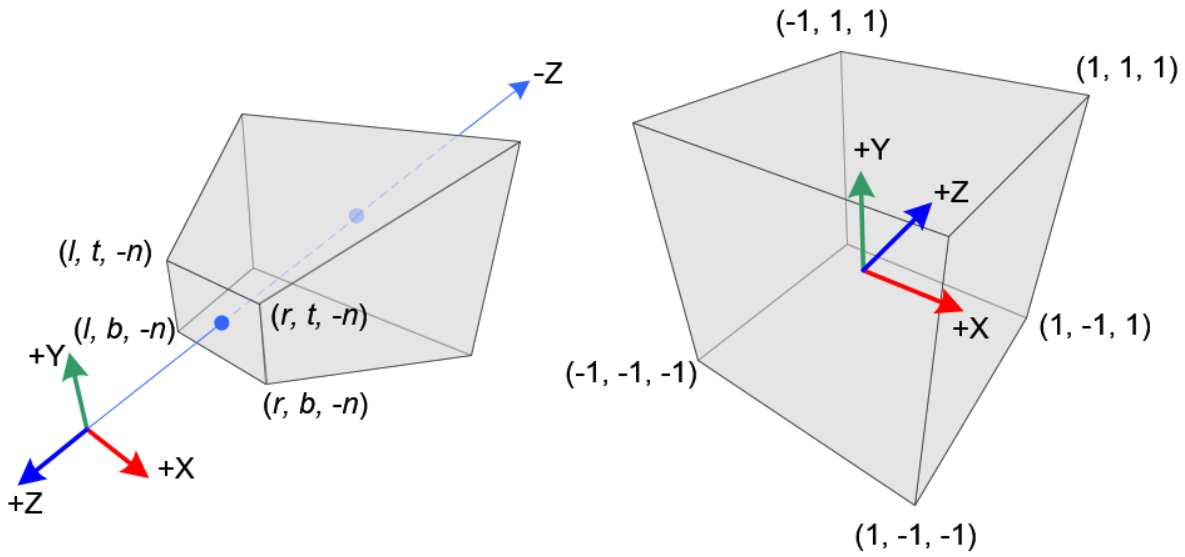
$$O = \begin{bmatrix} \frac{2}{right-left} & 0 & 0 & -\frac{right+left}{right-left} \\ 0 & \frac{2}{top-bottom} & 0 & -\frac{top+bottom}{top-bottom} \\ 0 & 0 & \frac{-2}{far-near} & -\frac{far+near}{far-near} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Projection perspective

Dans le cas d'une scène 3D, on souhaite avoir des "rayons" non plus parallèles mais qui convergent vers un point de fuite. Pour ce faire il est nécessaire que la position respective d'un vertex apparaisse proche du point de fuite lorsqu'il est éloigné de la position de la caméra dans l'espace.

On peut simuler cela très simplement en divisant x et y par la profondeur z, x' = x/z et y' = y/z. Pour être plus générique et permettre une adaptation au

Cependant, comme les données transitent via le "rasterizer" et que le GPU fonctionne en interne en NDC, la matrice est un peu plus complexe que cela. En effet, le "rasterizer" fonctionne comme on l'a vu par interpolation linéaire, or, la perception de la profondeur n'est pas linéaire (dépendante de z) mais hyperbolique (dépendante de 1/z). De plus, avant de diviser x et y par z, il faut s'assurer que les valeurs sont bien en NDC. C'est donc le rôle de cette matrice de projection perspective, voir plus bas pour le contenu de la matrice.

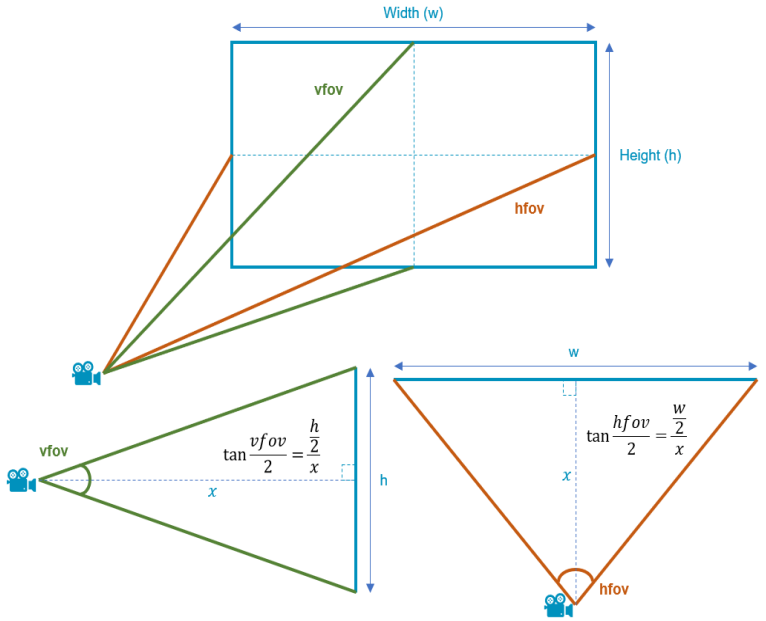


Les positions des plans "*near*" (proche) et "*far*" (lointain) vont nous permettre d'établir le rayon de la projection à travers une distance focale définie implicitement (distance entre l'oeil et la plan de projection ici le plan "*near*").

Les 6 plans forment alors une pyramide tronquée appelée "frustum":

$$P = \begin{bmatrix} \frac{2\ near}{right-left} & 0 & \frac{right+left}{right-left} & 0 \\ 0 & \frac{2\ near}{top-bottom} & \frac{top+bottom}{top-bottom} & 0 \\ 0 & 0 & -\frac{far+near}{far-near} & -\frac{2\ near\ far}{far-near} \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

Pour des raisons pratique, Il est toutefois recommandé d'utiliser la variante suivante, plus intuitive et plus simple à mettre en œuvre :



$$aspect_ratio = \frac{w}{h} = \frac{\tan\frac{hfov}{2}}{\tan\frac{vfov}{2}}$$

$$w\tan\frac{vfov}{2} = h\tan\frac{hfov}{2}$$

$$\frac{hfov}{2} = \tan^{-1}\frac{w\tan\frac{vfov}{2}}{h}$$

$$hfov = 2\tan^{-1}\left(\tan\frac{vfov}{2} \times \frac{w}{h}\right)$$

Plutôt que de renseigner la limite des 6 plans formant le frustum il est plus aisé de raisonner à l'aide d'un angle de vision (*field of view*, ou fov).

Comme on peut le voir il existe deux façons de calculer le fov : horizontalement (*hfov*) ou verticalement (*vfov*). Il est toujours possible de convertir d'un fov à l'autre. Dans ce document nous allons supposer un fov vertical. v*fov* correspond à l'angle (en radians) entre deux vecteurs formés respectivement par l'intersection d'un rayon partant de la position de l'œil vers la demi-hauteur positive du plan de projection (le plan *near* donc) ainsi que vers la demi-hauteur négative de ce même plan.

Notez que par l'application des théorèmes de Pythagore et Thalès on se rend compte visuellement que seul vfov/2 va être important dans le calcul de la distance focale x.

Dans la matrice ci-dessous 'f' correspond à 1 / tan(vfov/2) <=> cotan(vfov/2).

'aspect' est le ratio d'aspect (*aspect ratio*) qui indique la proportion entre l'axe des abscisses et l'axe des ordonnées du plan de projection. Ce qui correspond en pratique, ici, à longueur (*width*) divisée par la hauteur (*height*).

$$P = \begin{bmatrix} \frac{f}{aspect} & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & \frac{far+near}{near-far} & \frac{2\ near\ far}{near-far} \\ 0 & 0 & -1 & 0 \end{bmatrix} \text{ with } f = cotangent(\frac{fovy}{2})$$